

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа № 2

Выполнил:

Лесовой Никита

Группа:

J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Имитация работы с API". Для варианта 3 потребовалось хранить пользователей, туристические направления, сохранённые маршруты, поездку и заметки в JSON Server, выполнять запросы через Axios и загружать прогноз погоды из внешнего Open-Meteo API.

Ход работы

1. Подготовка API. В проект добавлены package.json и db.json. В package.json настроена команда start для запуска JSON Server, а db.json описывает коллекции users, destinations, savedRoutes, trips и notes. Каждая коллекция стала REST-ресурсом: например, список направлений доступен по адресу GET /destinations.

Листинг 1 — пример записи пользователя в db.json

```
"users": [  
  {  
    "id": "1",  
    "firstName": "Никита",  
    "lastName": "Лесовой",  
    "email": "nikita@example.com",  
    "password": "123456",  
    "city": "Санкт-Петербург"  
  },  
]
```

2. Общий Axios-клиент. Библиотека Axios подключается к HTML-страницам через CDN, а в файле js/api.js создан её экземпляр с baseUrl http://localhost:3000 и тайм-аутом пять секунд. Request-interceptor добавляет к запросам учебный Bearer-токен из localStorage. В этом же файле находятся функции чтения и сохранения текущего пользователя, форматирования цены и вывода понятного сообщения при сетевой ошибке.

Листинг 2 — создание Axios-клиента в js/api.js

```
const API_URL = "http://localhost:3000";

const api = axios.create({
  baseURL: API_URL,
  timeout: 5000
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("authToken");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});
```

3. Авторизация и регистрация. Файл js/auth.js обрабатывает формы входа и регистрации. Перед запросом проверяется встроенная HTML-валидация. При входе выполняется GET /users с параметром email и сравнивается пароль. При регистрации сначала проверяется отсутствие дубликата, затем создаётся новый пользователь запросом POST /users. После успешного ответа безопасная часть профиля и демонстрационный токен сохраняются в localStorage.

Листинг 3 — проверка пользователя при входе в js/auth.js

```
try {
  showMessage(result, "Проверяем данные...", "info");
  const response = await api.get("/users", {
    params: { email: form.elements.email.value.trim() }
  });
  const user = response.data.find(
    (item) => item.password === form.elements.password.value
  );

  if (!user) {
    showMessage(result, "Неверная почта или пароль.");
    return;
  }

  saveCurrentUser(user);
  showMessage(result, "Вход выполнен. Открываем личный кабинет...", "success");
  window.setTimeout(() => {
    window.location.href = "profile.html";
  }, 600);
} catch (error) {
  showMessage(result, getApiErrorMessage(error));
}
```

4. Работа страниц с данными. В `js/main.js` добавлены загрузка каталога, серверная фильтрация, отрисовка карточек и загрузка отдельного направления по его `id`. Страница профиля параллельно получает сохранённые маршруты и каталог через `Promise.all`. На странице поездки загружаются данные `trip` и `notes`, а добавление и удаление заметок выполняется запросами `POST` и `DELETE`.

5. Внешний прогноз и проверка. Для Open-Meteo создан отдельный `Axios`-запрос, потому что внешний API имеет другой адрес и не должен смешиваться с `JSON Server`. По координатам направления запрашиваются текущая температура и прогноз на несколько дней. Проект проверен командами `npm install` и `npm start`: после запуска API в браузере проверены каталог, фильтры, вход тестового пользователя, сохранение маршрута, заметки и отображение сообщения при недоступном сервере.

Листинг 4 — запрос прогноза у Open-Meteo в `js/main.js`

```
const response = await axios.get("https://api.open-meteo.com/v1/forecast", {
  timeout: 6000,
  params: {
    latitude: destination.latitude,
    longitude: destination.longitude,
    current: "temperature_2m,weather_code,wind_speed_10m",
    daily: "weather_code,temperature_2m_max,temperature_2m_min",
    forecast_days: 4,
    timezone: "auto"
  }
});
```

Вывод

В результате статический интерфейс был соединён с REST API и внешним сервисом погоды. Пользовательские сценарии стали работать с данными из `db.json`, а не с заранее записанными HTML-блоками. Реализация является учебной: пароли находятся в `db.json` открытым текстом, а `demo-token` не проверяется настоящим сервером, поэтому такую авторизацию нельзя переносить в `production` без серверной защиты.